



MACHINE LEARNING IN BIOINFORMATICS

Part 3: Clustering

František Mráz
KSVI MFF UK

From Density Estimation to Clustering



- Types of methods for density estimation
 1. **Parametric:** Assume a single model for $p(x | C_i)$ (previous part)
 2. **Semiparametric:** $p(x | C_i)$ is a mixture of densities.
Multiple possible explanations/prototypes:
Different handwriting styles, accents in speech
 3. **Nonparametric:** No model; „data speaks for itself“
- Clustering can be considered as a type of density estimation
- Grouping and partitioning data are two powerful approaches for discovering relevant biological regulations which can be used in late hypothesis verification.
- Clustering in gene expression data to reveal gene functions, signaling pathways, and cellular processes categories

Types of clustering



- A. Grouping data:** data relationship reconstruction, i.e., exploring how data are clustered through a learning process
- B. Partitioning data:** to discover hidden data structure through a learning process
- Covered methods:
 1. K -means algorithm
 2. Fuzzy C -means algorithm
 3. Mixture models
 4. Hierarchical clustering
 5. Density based clustering



A nonparametric method

K-MEANS CLUSTERING ALGORITHM

Machine Learning in Bioinformatics

5

A nonparametric method

FUZZY c -MEANS CLUSTERING

Machine Learning in Bioinformatics

19

A parametric model

MIXTURE MODELS

Machine Learning in Bioinformatics

23

HIERARCHICAL CLUSTERING

Machine Learning in Bioinformatics

36

DBScan

DENSITY BASED CLUSTERING

Machine Learning in Bioinformatics

47

AFTER CLUSTERING

Machine Learning in Bioinformatics

49



A nonparametric method

K-MEANS CLUSTERING ALGORITHM

1. k-Means Clustering (a nonparametric algorithm)



- Find k reference vectors (prototypes / codebook vectors / codewords) which best represent data
- Reference vectors, $m_j, j = 1, \dots, k$
- For representing x_t use the nearest (most similar) reference m_i such that:

$$\|x_t - m_i\| = \min_j \|x_t - m_j\|$$

- Minimize reconstruction error

Error, not the expected value

$$E(\{m_i\}_{i=1}^k | \mathbf{X}) = \sum_{t=1}^N \sum_{i=1}^k b_t^{(i)} \|x_t - m_i\|^2$$

- $b_t^{(i)} = \begin{cases} 1 & \text{if } \|x_t - m_i\| = \min_j \|x_t - m_j\|, m_i \text{ represents } x_t \\ 0 & \text{otherwise} \end{cases}$

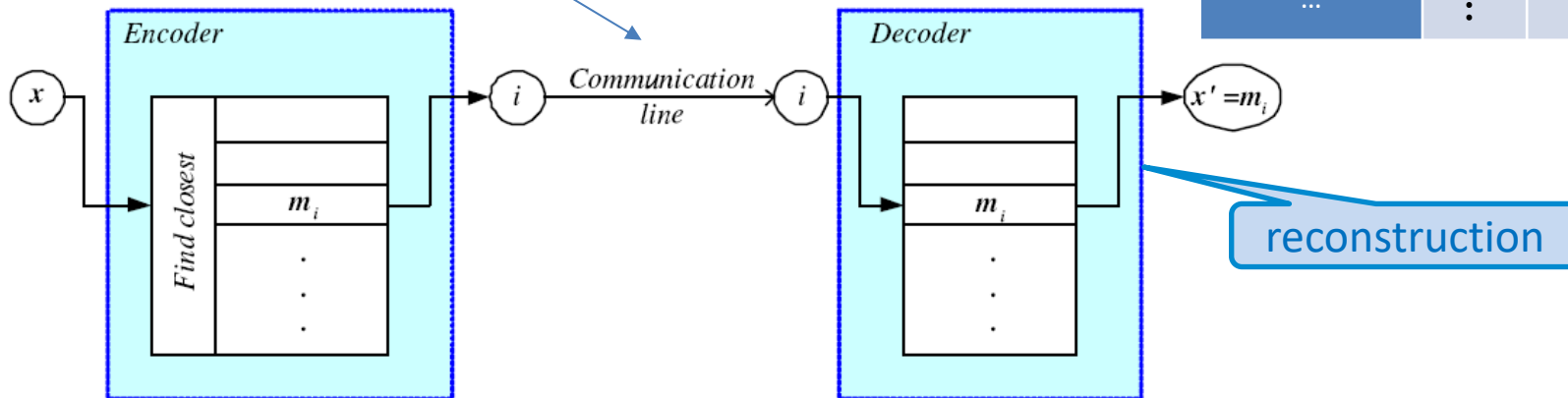
Representation



$$b_t^{(i)} = \begin{cases} 1 & \text{if } \|x_t - m_i\| = \min_j \|x_t - m_j\| \\ 0 & \text{otherwise} \end{cases}$$

- $b_t^{(i)}$ is a one-hot-encoding for representants

i	$b_t^{(1)}$	$b_t^{(2)}$	$b_t^{(3)}$
Observation 1	1	0	0
Observation 2	0	0	1
Observation 3	0	1	0
Observation 4	1	0	0
...	$\vdots$	$\vdots$	$\vdots$



k-means Clustering



Input: dataset $\mathbf{X} = \{x_t\}_{t=1}^N$, number of clusters $k > 1$

Initialize m_i , $i = 1, \dots, k$, for example by k random x from $\mathbf{X}$

Repeat

for all $x_t \in \{x_t\}_{t=1}^N$

$$\text{Compute } b_t^{(i)} = \begin{cases} 1 & \text{if } \|x_t - m_i\| = \min_j \|x_t - m_j\| \\ 0 & \text{otherwise} \end{cases}$$

for all $m_i, i = 1, \dots, k$

$$m_i = \frac{\sum_t x_t b_t^{(i)}}{\sum_t b_t^{(i)}}$$

Until m_i converge

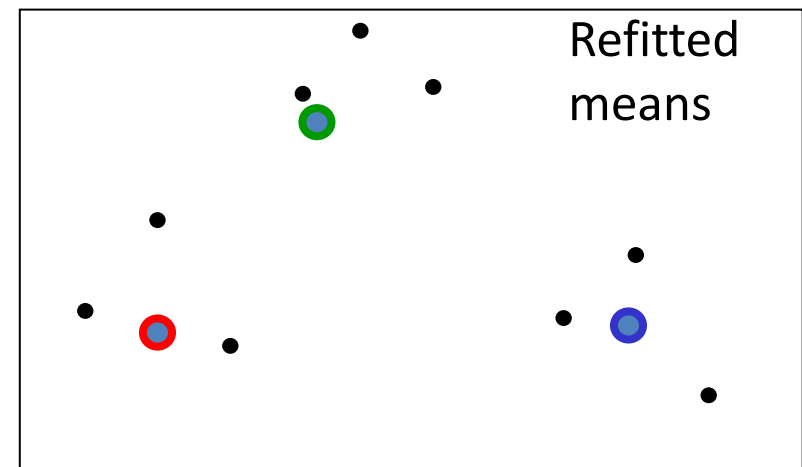
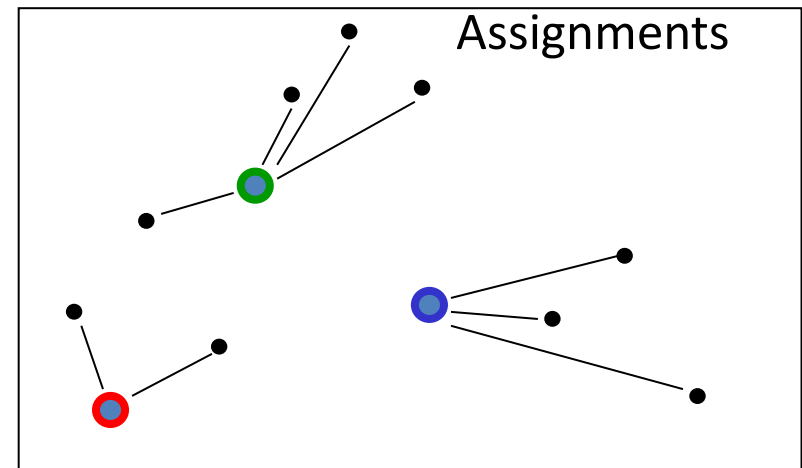
The k -Means Algorithm

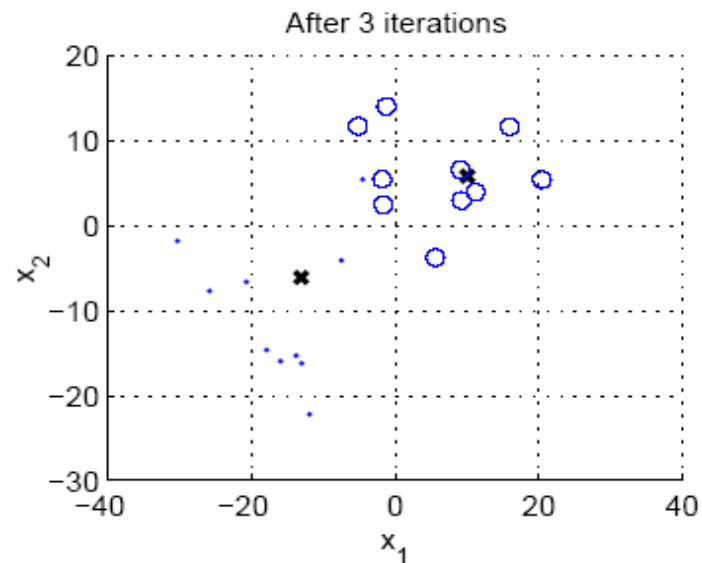
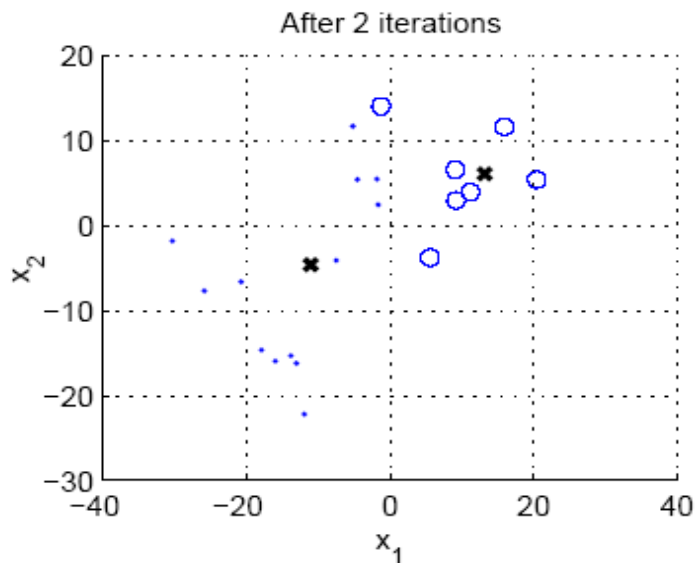
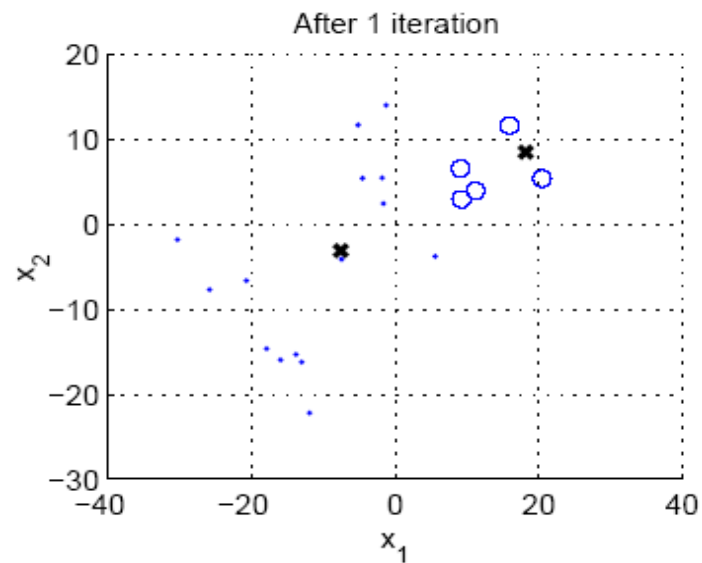
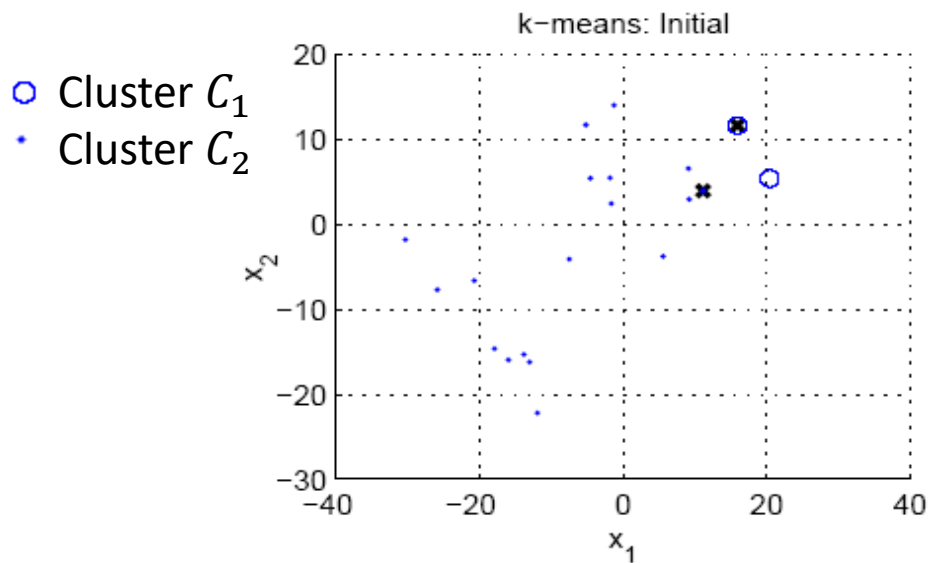


- Assume
 - the data lives in a Euclidian space.
 - we want k classes.
 - we start with randomly located cluster centers

The algorithm alternates between two steps:

1. **Assignment step:** Assign each datapoint to the closest cluster.
2. **Refitting step:** Move each cluster center to the center of gravity of the data assigned to it.





Why k -Means Converges



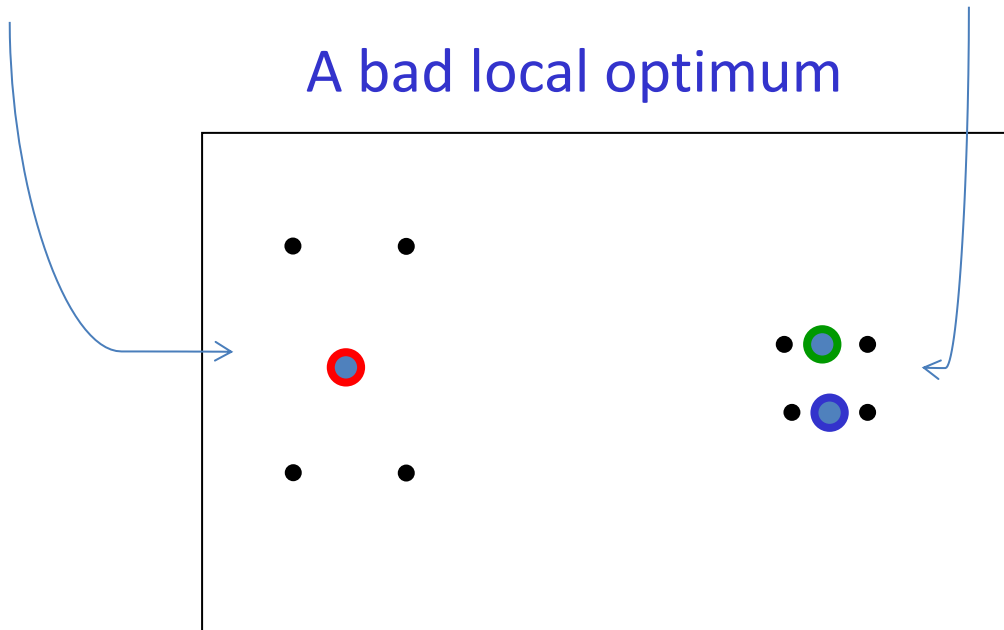
- Whenever an assignment is changed, the sum squared distances of datapoints from their assigned cluster centers is reduced.
- Whenever a cluster center is moved the sum squared distances of the datapoints from their currently assigned cluster centers is reduced.
- **Test for convergence:** If the assignments do not change in the assignment step, we have converged.

Local Minima



- k -means can get stuck at local minima.
- We could try many random starting points
- We could try non-local split-and-merge moves: Simultaneously split a big cluster into two and/or merge two nearby clusters.

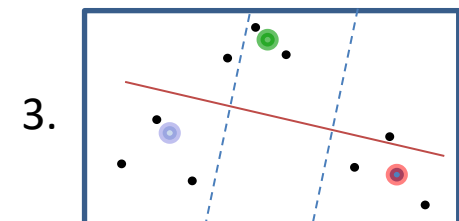
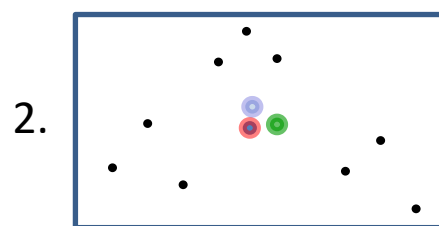
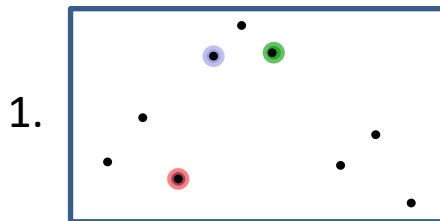
A bad local optimum



Initialization



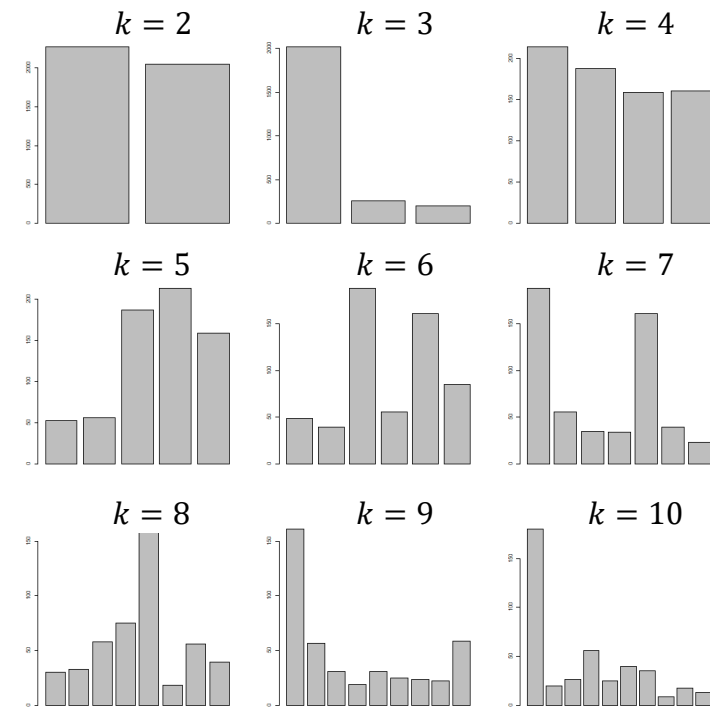
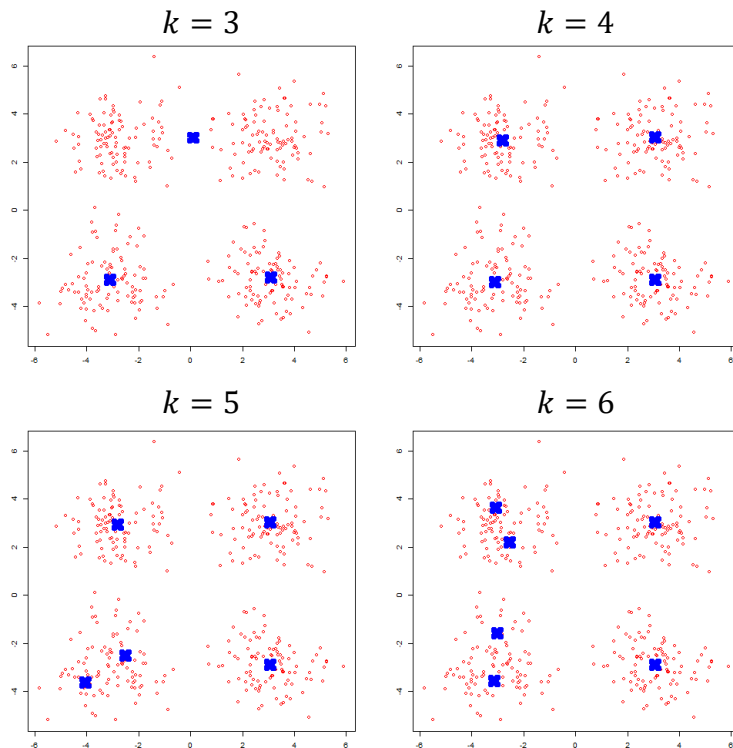
- Final m_i depends on the initial value of m_i
- Methods of initialization:
 1. Random k instances from the input
 2. Compute mean of the data and add k small random vectors to the mean – these will be the initial $m_1, \dots, m_k$
 3. Compute the first principal component, divide its range into k equal intervals and based on that partition the data into k groups



How many centers?

Within-cluster diversity differences are the smallest for $k = 2$ and $k = 4$

- Introduce a **within-cluster diversity** and compare it for different values of k



The vertical axes represent the within-cluster diversity
= within-cluster sum of squares

How many centers?



- Statistical measures
 1. Akaike information criterion (AIC)
 2. Schwarz's Bayesian information criterion (BIC)

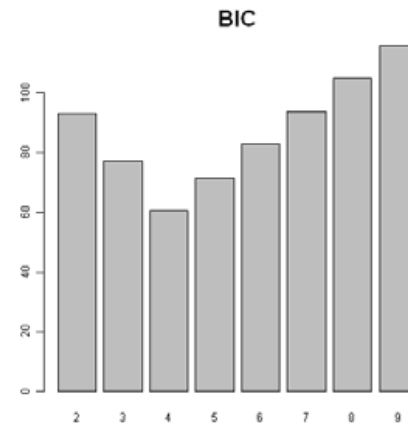
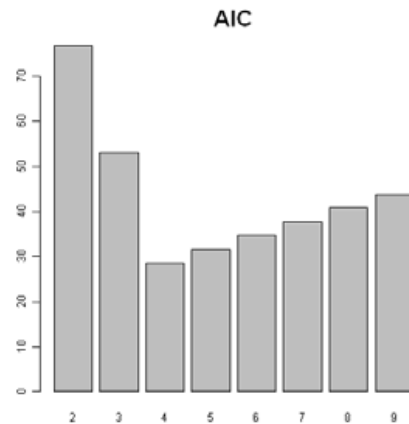
$$\text{Minimize:} \quad -2 \log \mathcal{L} + \lambda M = -\log(\mathcal{L}^2 \cdot 2^{\lambda M})$$

- $\mathcal{L}$ is the model likelihood
- M is the number of model parameters
- λ is a constant:
 - $\lambda = 2$ for AIC
 - $\lambda = \log N$ for BIC, where N is the number of data points.
- Both criteria are minimized to select the best model.

Information Criteria



$$-2 \log \mathcal{L} + 2M$$



$$-2 \log \mathcal{L} + M \log N$$

$$-2 \log \mathcal{L} + \lambda M$$

- model likelihood $\mathcal{L}$ must be provided
 - In a k -means cluster model, the likelihood can be calculated by assuming that the data points in **each** cluster follow a multivariate Gaussian distribution

Clustering criterions



When we know the correct assignments C^* to the clusters

- Compare obtained clustering C to the true clustering C^*
- **Rand Index (RI , `rand_score` in `scikit-learn`):** measure the similarity of predicted and true assignment
 - a the number of pairs that are assigned to the same cluster by both C and C^*
 - b the number of pairs that are assigned to different clusters by C and C^*

$$RI = \frac{a + b}{\binom{N}{2}}$$

- Easy interpretation, range $\langle 0; 1 \rangle$
- Random clustering has often high $RI \neq 0$, the perfect assignment has $RI = 1$
- **Adjusted RI** (`adjusted_rand_score` in `scikit-learn`):
$$ARI = \frac{RI - E[RI]}{\max(RI) - E[RI]} \quad \text{– range } \langle -0.5; 1 \rangle, \text{ random clustering has zero } ARI$$

Clustering criterions



When the correct assignment to clusters is not known

- **Silhouette Coefficient** (`silhouette_score` in `scikit-learn`):
 - a = the mean distance between a sample and all other points in the same cluster.
 - b = the mean distance between a sample and all other points in the *next nearest cluster*.
 - The Silhouette Coefficient s for a single sample is then given as:

$$s = \frac{b - a}{\max(a, b)}$$

- range $\langle -1; 1 \rangle$



A nonparametric method

FUZZY c -MEANS CLUSTERING

2. Fuzzy c -Means Clustering



- When analyzing gene expression data from microarrays, genes with similar expression patterns can be grouped into the same cluster; however, one gene may be acted on by more than one transcription factor, and one gene may encode a protein that has more than one function
- Fuzzy clustering allows genes to belong to more than one cluster, it allows for the identification of genes that are conditionally co-regulated or co-expressed
- One cluster may have a responsibility of 0.7 for a datapoint and another may have a responsibility of 0.3.
 - Allows a cluster to use more information about the data in the refitting step.
 - What happens to our convergence guarantee?
 - How do we decide on the soft assignments?

Fuzzy c -Means



- Instead of minimizing

$h_t^{(i)}$ is the soft assignment of x_t to the cluster i

$$\sum_{t=1}^N \sum_{i=1}^k b_t^{(i)} \|x_t - m_i\|^2$$

Another parameter of the model

- We minimize

$$\sum_{t=1}^N \sum_{i=1}^k \left(h_t^{(i)}\right)^m \|x_t - m_i\|^2 \quad \text{for some constant } m \geq 1,$$

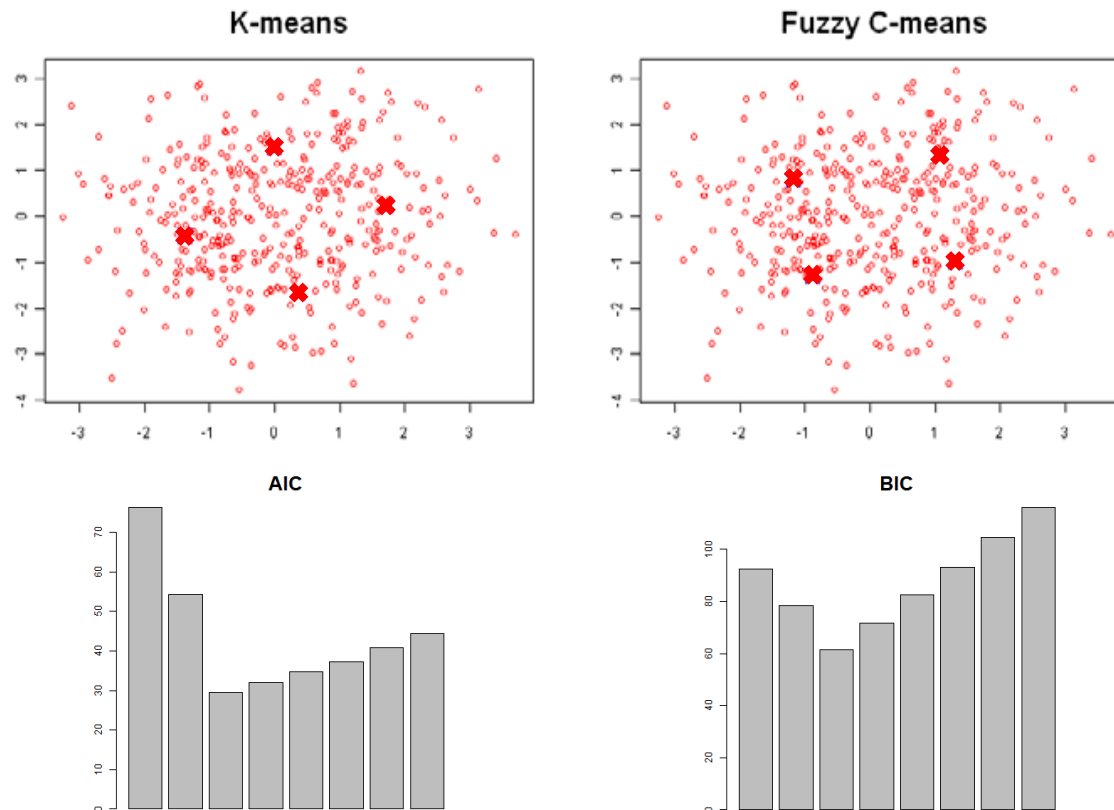
$$\text{where } m_i = \frac{\sum_t x_t \left(h_t^{(i)}\right)^m}{\sum_t \left(h_t^{(i)}\right)^m} \quad \text{and} \quad h_t^{(i)} = \frac{1}{\sum_{j=1}^k \left(\frac{\|x_t - m_i\|}{\|x_t - m_j\|}\right)^{\frac{2}{m-1}}}$$

- Fuzzy c -means algorithm benefits from its continuous (hence soft) membership function from which the centers of clusters can be more accurately estimated
- In the name *Fuzzy c -means algorithm* used in literature the c refers to the number of clusters. In our case it would be better **Fuzzy k -means algorithm**

Fuzzy c -Means



- Source data are generated from normal distributions with centers $(-1, -1)$, $(-1, 1)$, $(1, -1)$, $(1, 1)$ with standard deviation 1



Information
criteria for
fuzzy c -means



A parametric model

MIXTURE MODELS

Mixture Models



- Both the k -means algorithm and the fuzzy c -means algorithm are designed to detect cluster structure with cluster densities distributed homogeneously in all dimensions, i.e., the volume of each cluster is symmetrical with respect to the cluster center
- Such assumption is relaxed in mixture models

Mixture Models



- Mixture densities

$$p(x) = \sum_{i=1}^k p(x|G_i)P(G_i)$$

- where

G_i are the components/groups/clusters,

$P(G_i)$ mixture proportions (priors), $\sum_{i=1}^k P(G_i) = 1$, $P(G_i) \geq 0$

$p(x|G_i)$ component densities

- Gaussian mixture where $p(x|G_i) \sim \mathcal{N}(\mu_i, \Sigma_i)$ and parameters $\Phi = \{P(G_i), \mu_i, \Sigma_i\}_{i=1}^k$ must be estimated from the unlabeled samples (unsupervised learning) $\mathbf{X} = \{x_t\}_{t=1}^N$
- In the density estimation we maximize likelihood of the sample (the observed data)

Expectation- Maximization (EM)



- In k -means we minimized the total reconstruction error; original data are $\mathbf{X} = \{x_t\}_{t=1}^N$
- Next, we will look for the component density parameters that maximize the likelihood of the sample
- Log-likelihood with a mixture model

In general likelihood $\mathcal{L}(A | B) = p(B | A)$ is a function of B

Parameter
vector of
the model

$$\mathcal{L}(\Phi | \mathbf{X}) = \log \prod_{t=1}^N p(x_t | \Phi) = \sum_t \log \sum_{i=1}^k p(x_t | G_i) P(G_i)$$

Maximization
cannot be solved
analytically

- Assume hidden variables Z , which when known, make optimization much simpler
- Complete (log)likelihood, $\mathcal{L}_c(\Phi | \mathbf{X}, Z)$, in terms of $\mathbf{X}$ and Z
- Incomplete (log)likelihood, $\mathcal{L}(\Phi | \mathbf{X})$, in terms of $\mathbf{X}$

E- and M-steps



Iterate the two steps

1. E-step: Estimate Z given $\mathbf{X}$ and current Φ

$$\text{E-step: } Q(\Phi | \Phi^{(l)}) = E[\mathcal{L}_c(\Phi | \mathbf{X}, Z) | \mathbf{X}, \Phi^{(l)}]$$

2. M-step: Find new Φ' given Z , $\mathbf{X}$, and old Φ :

$$\text{M-step: } \Phi^{(l+1)} = \arg \max_{\Phi} Q(\Phi | \Phi^{(l)})$$

An increase in Q (the expected (log)likelihood) increases incomplete (log)likelihood [Dempster, Laird, and Rubin (1977)]

$$\mathcal{L}(\Phi^{(l+1)} | \mathbf{X}) \geq \mathcal{L}(\Phi^{(l)} | \mathbf{X})$$

E- and M-steps



- In the case of mixtures, the hidden variables are the sources of observations – which observation belongs to which component.
- If these were given (e.g., as class labels in a supervised setting) we would know which parameters to adjust to fit that data point.
- The EM algorithm works as follows:
 1. in the E-step we estimate these labels given our current knowledge of components, and
 2. in the M-step we update our component knowledge given the labels estimated in the E-step.
- These two steps are the same as the two steps of *k*-means
 1. calculation of $b_t^{(i)}$ (E-step), and
 2. Re-estimation of m_i (M-step).

EM in Gaussian Mixtures



Indicator variables

- $Z = \{z_t^{(i)}\}_{t=1}^N$, $z_t^{(i)} = 1$ if x_t belongs to G_i , 0 otherwise (it corresponds to labels $r_t^{(i)}$ of supervised learning); assume $p(x|G_i) \sim \mathcal{N}(\mu_i, \Sigma_i)$

$$\mathcal{L}_c(\Phi | \mathbf{X}, Z) = \log \prod_t p(x_t, z_t | \Phi)$$

x_t are iid

$$P(A, B | C) = P(B | C)P(A | B, C)$$

$$= \sum_t \log p(x_t, z_t | \Phi) = \sum_t \log P(z_t | \Phi) p(x_t | z_t, \Phi)$$

$$= \sum_t [\log P(z_t | \Phi) + \log p(x_t | z_t, \Phi)]$$

$$\sum_{t=1}^N \sum_{i=1}^k z_t^{(i)} [\log P(G_i) + \log p(x_t | G_i)]$$

$z_t^{(i)}$ is 0/1

$$P(z_t) = \prod_{i=1}^k P(G_i)^{z_t^{(i)}}$$

$$\log P(z_t) = \sum_{i=1}^k z_t^{(i)} \log P(G_i)$$

EM in Gaussian Mixtures



Indicator variables

- $Z = \{z_t^{(i)}\}_{i=1}^N$, $z_t^{(i)} = 1$ if x_t belongs to G_i , 0 otherwise (it corresponds to labels $r_t^{(i)}$ of supervised learning); assume $p(x|G_i) \sim \mathcal{N}(\mu_i, \Sigma_i)$
- **E-step:**

$$\begin{aligned} Q(\Phi | \Phi^{(l)}) &\equiv_{def} E[\log P(\mathbf{X}, Z) | \mathbf{X}, \Phi^{(l)}] \\ &= E[\mathcal{L}_c(\Phi | \mathbf{X}, Z) | \mathbf{X}, \Phi^{(l)}] \\ &= \sum_{t=1}^N \sum_{i=1}^k E[z_t^{(i)} | \mathbf{X}, \Phi^{(l)}] [\log P(G_i) + \log p(x_t | G_i, \Phi^{(l)})] \end{aligned}$$

- For computing $E[z_t^{(i)} | \mathbf{X}, \Phi^{(l)}]$ see the next slide

EM in Gaussian Mixtures



iid =
independent
identically
distributed

$$E \left[z_t^{(i)} \mid \mathbf{X}, \Phi^{(l)} \right] = E \left[z_t^{(i)} \mid x_t, \Phi^{(l)} \right]$$

x_t are iid

$$= P \left(z_t^{(i)} = 1 \mid x_t, \Phi^{(l)} \right)$$

$z_t^{(i)}$ is a 0/1
random variable

$$= \frac{p \left(x_t \mid z_t^{(i)} = 1, \Phi^{(l)} \right) P \left(z_t^{(i)} = 1 \mid \Phi^{(l)} \right)}{p \left(x_t \mid \Phi^{(l)} \right)}$$

$$\begin{aligned} P(A \mid B, C)P(B \mid C)P(C) &= P(B \mid A, C)P(A \mid C)P(C) \\ P(A \mid B, C)P(B \mid C) &= P(B \mid A, C)P(A \mid C) \end{aligned}$$

$$\begin{aligned} &= \frac{p \left(x_t \mid G_i, \Phi^{(l)} \right) P(G_i)}{\sum_j p \left(x_t \mid G_j, \Phi^{(l)} \right) P(G_j)} \\ &= P \left(G_i \mid x_t, \Phi^{(l)} \right) =: h_t^{(i)} \end{aligned}$$

New notation

Notation:

$P(X)$ probability of X when X is discrete

$p(X)$ probability density function when X is continuous

EM in Gaussian Mixtures



- **M-step:**

$$\begin{aligned}\Phi^{(l+1)} &= \arg \max_{\Phi} Q(\Phi | \Phi^{(l)}) = \arg \max_{\Phi} E[\mathcal{L}_c(\Phi | \mathbf{X}, Z) | \mathbf{X}, \Phi^{(l)}] \\ &= \arg \max_{\Phi} \sum_t \sum_i E[z_t^{(i)} | \mathbf{X}, \Phi^{(l)}] \cdot [\log P(G_i) + \log p(x_t | G_i, \Phi^{(l)})] \\ &= \arg \max_{\Phi} \sum_t \sum_i h_t^{(i)} \log P(G_i) + \sum_t \sum_i h_t^{(i)} \cdot \log p(x_t | G_i, \Phi^{(i)})\end{aligned}$$

- Solution using Lagrange multiplier method

$$P(G_i) = \frac{\sum_t h_t^{(i)}}{N} \quad m_i^{(l+1)} = \frac{\sum_t h_t^{(i)} x_t}{\sum_t h_t^{(i)}}$$

Use estimated labels in place of unknown labels

$$\mathbf{S}_i^{(l+1)} = \frac{\sum_t h_t^{(i)} (x_t - m_i^{(l+1)}) (x_t - m_i^{(l+1)})^T}{\sum_t h_t^{(i)}}$$

EM in Gaussian Mixtures



- For Gaussian components in the E-step

$$h_t^{(i)} = \frac{p(G_i) |\Sigma_i|^{-\frac{1}{2}} \exp \left[-\frac{1}{2} (x_t - m_i)^T \Sigma_i^{-1} (x_t - m_i) \right]}{\sum_j p(G_j) |\Sigma_j|^{-\frac{1}{2}} \exp \left[-\frac{1}{2} (x_t - m_j)^T \Sigma_j^{-1} (x_t - m_j) \right]}$$

- Overall steps:

1. EM is initialized by k -means

- it estimates m_i
- we estimate Σ_i and compute $p(G_i)$ as

$$\frac{\sum_t b_t^{(i)}}{N}$$

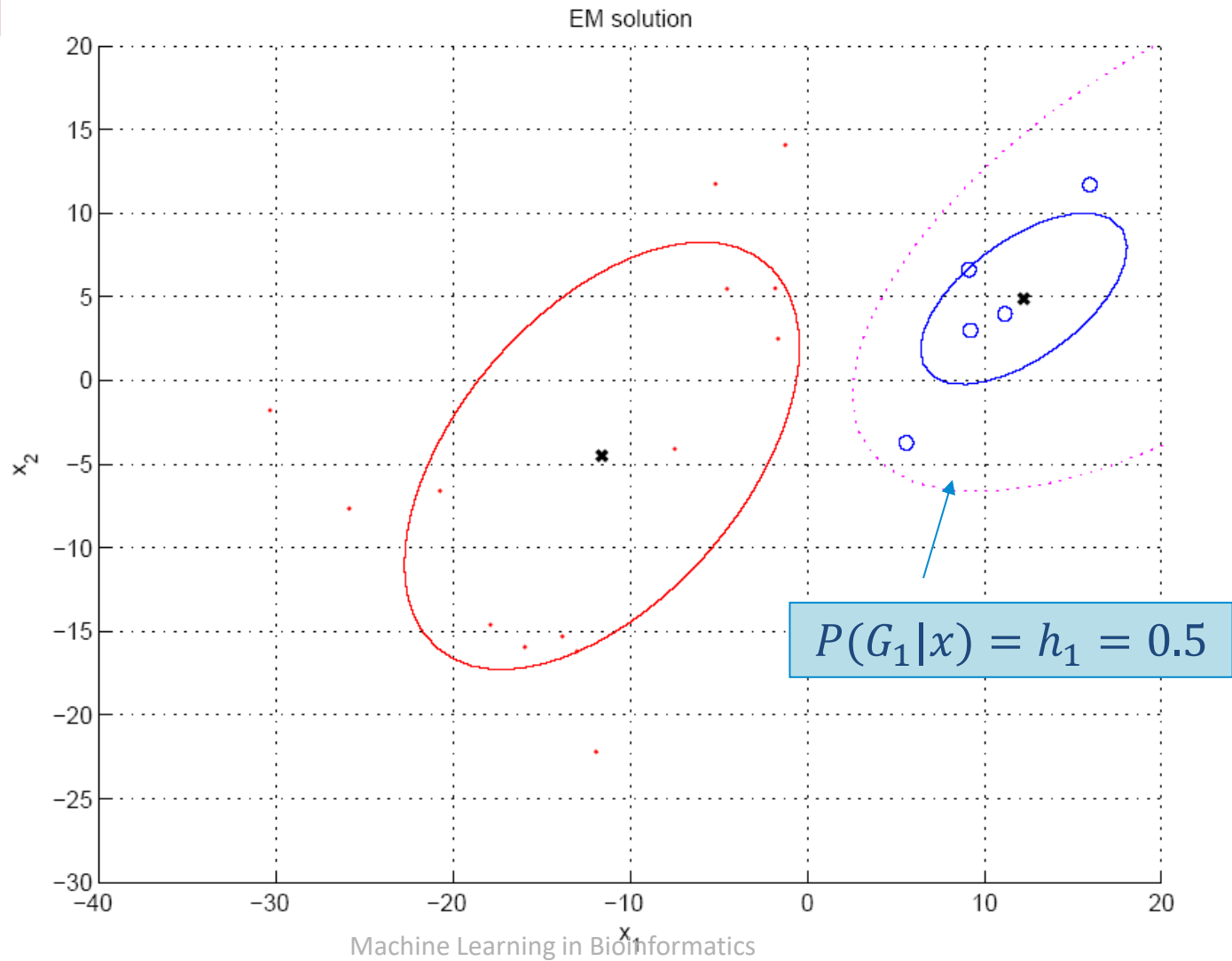
2. Run EM by alternating E- and M-steps

- By making a few simplifying assumptions (the same diagonal covariance matrix for all clusters) we obtain that k -means clustering is a special case of EM applied to Gaussian mixtures where inputs are assumed to be independent with equal and shared variances, all components have equal priors, and labels are hardened

EM in Gaussian Mixtures



- k -means thus pave the input density with circles, whereas EM in the general case uses ellipses of arbitrary shapes, orientations, and coverage proportions



Mixtures of Latent Variable Models



- Using full covariance matrices with Gaussian mixtures ($p(x|G_i) \sim \mathcal{N}(\mu_i, \Sigma_i)$), even if there is no singularity, can cause overfitting if the input dimensionality is high and the sample is small → the solution is to **regularize** clusters to decrease the number of parameters:
 1. Assuming a **common covariance matrix** – may not help, as clusters may have different shapes.
 2. Assuming **diagonal covariance matrices** – is even more riskier because it removes all correlations.
 3. Use **PCA/FA** to decrease dimensionality: Mixtures of PCA/FA

$$p(x_t|G_i) = \mathcal{N}(m_i, \mathbf{V}_i \mathbf{V}_i^T + \Psi_i)$$

Factor loadings of G_i

Specific variance of G_i

We can use EM to learn $\mathbf{V}_i$ (Ghahramani and Hinton, 1997; Tipping and Bishop, 1999)



HIERARCHICAL CLUSTERING

Hierarchical Clustering



- Clusters based on similarities/distances
- Distance measure between instances x_r and x_s
Minkowski (L_p) (Euclidean for $p = 2$)

$$d_m(x_r, x_s) = \left[\sum_{j=1}^d (x_{rj} - x_{sj})^p \right]^{\frac{1}{p}}$$

City-block distance

$$d_{cb}(x_r, x_s) = \sum_{j=1}^d |x_{rj} - x_{sj}|$$

Agglomerative Clustering



- Start with N groups each with one instance and merge two closest groups at each iteration
- Distance between two groups G_i and G_j :

- Single-link:

$$d(G_i, G_j) = \min_{x_r \in G_i, x_s \in G_j} d(x_r, x_s)$$

- Complete-link:

$$d(G_i, G_j) = \max_{x_r \in G_i, x_s \in G_j} d(x_r, x_s)$$

- Average-link: *the average of distances between all pairs*
- centroid distance: *the distance between the centroids*

Agglomerative Clustering

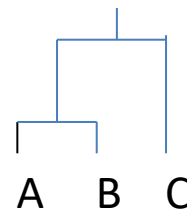
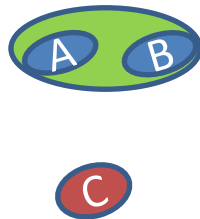


- **Input:** dataset $\mathbf{X} = \{x_t\}_{t=1}^N$ and a distance measure $d(G, G')$ for computing distance between clusters
- **Output:** a hierarchical clustering – a tree
- **Algorithm:**
 1. Put each x_t into its own cluster $\{x_t\}$ on a working list W
 2. Repeat
 - Find two clusters G_i, G_j in W that are most similar (have the lowest distance according to $d(\cdot, \cdot)$)
 - Remove both G_i, G_j from W , insert $G_i \cup G_j$ into WUntil W contains a single cluster
- The hierarchical clustering comprises all clusters that appeared during the run of the algorithm

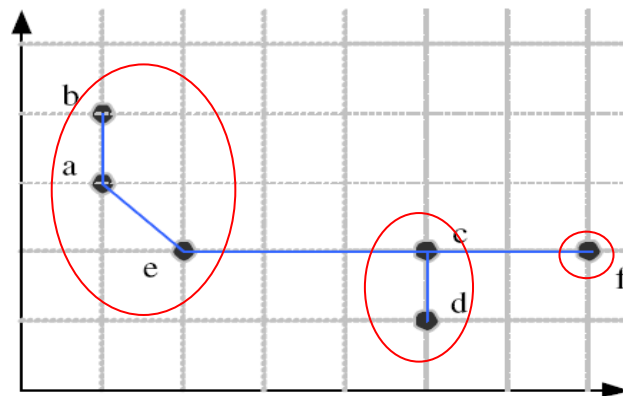
Dendrograms and Monotonicity



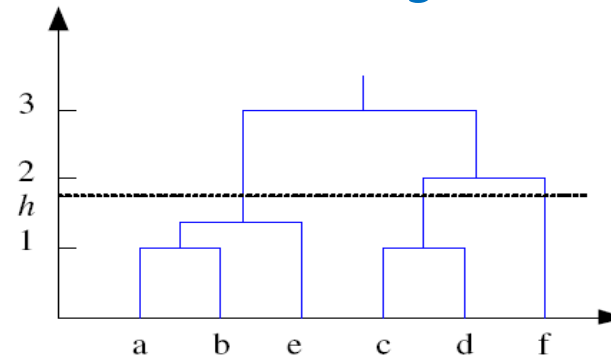
- Each $d(\cdot, \cdot) \in \{\text{Single-link, complete-link, average-link distance}\}$ dissimilarity measures satisfies
 - Let A, B and C be clusters, then
$$d(A, B) < d(A, C) \text{ and } d(A, B) < d(B, C) \text{ implies } d(A, B) < d(A \cup B, C)$$
- Note:** Centroid method is often used in genomics, but does not satisfy this; What are the consequences?
- Hence, when two clusters are merged during agglomerative clustering, their distance is not smaller than the distance of clusters in all previous merges
- Dendrograms are often drawn so that the **height** h of a node (merge) corresponds to the distance between the merged nodes (clusters)



Example: Single-Link Clustering

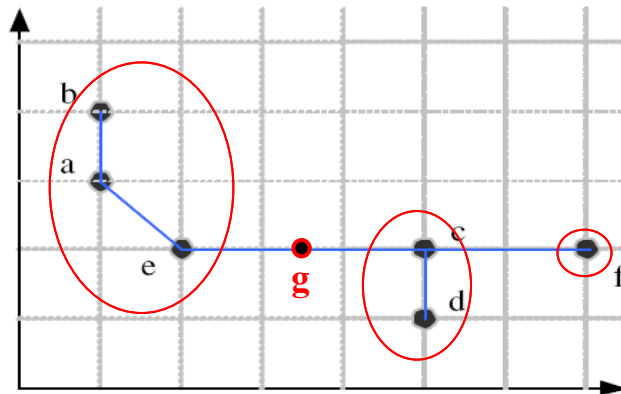


Dendrogram

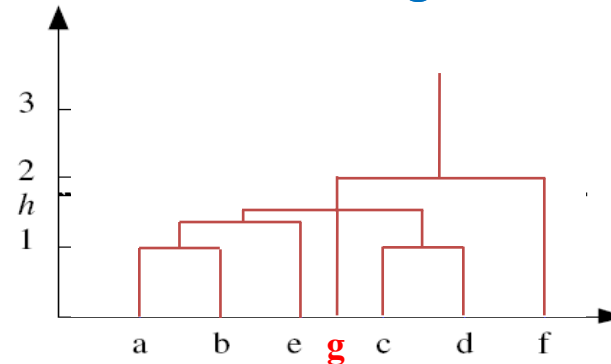


- In **single-link clustering**: Two instances are grouped together at level h if the distance between them is less than h , or if there is an intermediate sequence of instances between them such that the distance between consecutive instances is less than h . *E.g., if there would be a node in the middle between e and c , a node with three branches will be created at level 1.5*
- In the *complete-link* method, all instances in a group have a distance less than h **among** them

Example: Single-Link Clustering – Added g

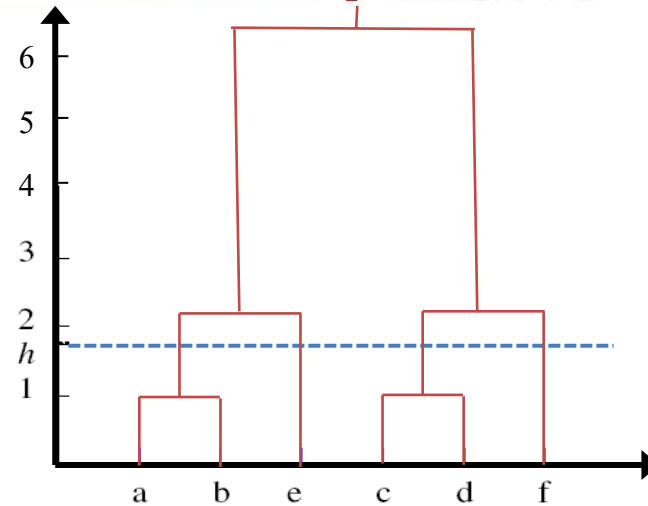
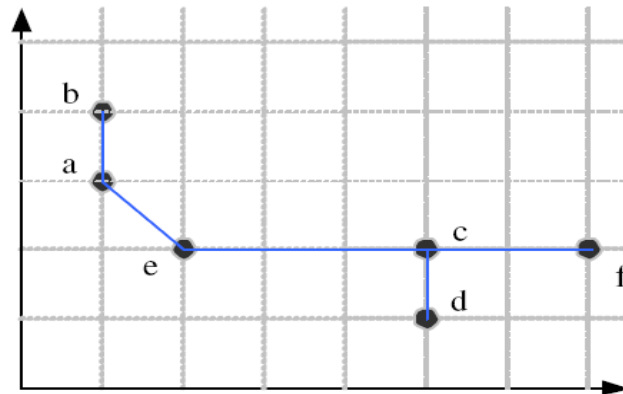


Dendrogram



- In **single-link clustering**: Two instances are grouped together at level h if the distance between them is less than h , or if there is an intermediate sequence of instances between them such that the distance between consecutive instances is less than h . E.g., **if there would be a node in the middle between e and c , a node with three branches will be created at level 1.5**
- In the *complete-link* method, all instances in a group have a distance less than h **among** them

Example: Complete-Link Clustering



- In **single-link clustering**: Two instances are grouped together at level h if the distance between them is less than h , or if there is an intermediate sequence of instances between them such that the distance between consecutive instances is less than h . E.g., if there would be a node in the middle between e and c , a node with three branches will be created at level 1.5
- In the *complete-link* method, all instances in a group have a distance less than h **among** them

Choosing k



- Defined by the application, e.g., image quantization
- Plot data (after PCA) and check for clusters
- Incremental (leader-cluster) algorithm: Add one at a time until “elbow” (reconstruction error / log likelihood/ intergroup distances)
- Manually check for meaning



DBScan

DENSITY BASED CLUSTERING

DBSCAN



- **Density-Based Spatial Clustering of Applications with Noise**
- Finds **core** samples of high density and expands clusters from them. Good for data which contains clusters of similar density.
- Using two parameters **min_samples** and **eps**, observations are partitioned:
 - **Core samples:** such that there exist **min_samples** other samples within the distance **eps**, which are defined as neighbors of the core sample.
 - **Neighbors:** other samples within the distance at most **eps**
 - **Outliers**
- **A cluster**
 - find a core sample, recursively find all its neighbors that are core samples
 - add non-core samples that are neighbors of a core sample

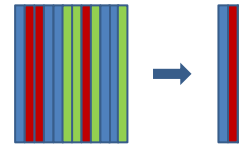


AFTER CLUSTERING

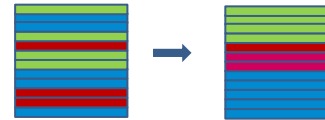
After Clustering



- Dimensionality reduction methods find correlations between features and group features



- Clustering methods find similarities between instances and group instances

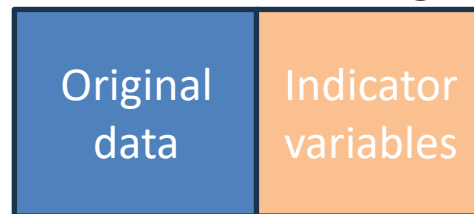


- Clustering allows knowledge extraction through
 - number of clusters,*
 - prior probabilities,*
 - cluster parameters, e.g., center, range of features.*
- Example: customer relationship management (CRM)
 - First clustering = customer segmentation
 - Then different strategies for different types of customers

Clustering as Preprocessing



- Unsupervised clustering can be applied on all available data and then followed by supervised learning
 - Estimated group labels $h^{(j)}$ (soft) or $b^{(j)}$ (hard) may be seen as the dimensions of a new k -dimensional space, where we can then learn our discriminant or regressor.



- Local representation (only one $b^{(i)}$ is 1, all others are 0; only few $h^{(j)}$ are nonzero) vs Distributed representation (After PCA; all $z^{(j)}$ are nonzero)
- Obtaining labels for supervised learning can be costly – it can be done in a more directed way after knowing the clusters, e.g., label propagation